SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces **TreasurAI,** an AI-powered financial decision system designed to assist corporate finance teams in optimizing foreign exchange (FX) decisions, treasury operations, and supplier negotiations using real-time data and ILMU AI GLM reasoning capabilities.

The system transforms traditional manual financial decision-making into an intelligent, data-driven, and explainable AI-assisted workflow.

1.1. Purpose:

The system analysis document defines the technical architecture, AI integration strategy, and system design decisions behind TreasurAI. It provides a structured overview of how financial data, market intelligence, and AI reasoning are combined to generate optimized corporate financial decisions.

So, the key elements of system that has been covered are as followed:

1. **Architecture**:

   TreasurAI is designed as a cloud-based AI financial decision platform with modular components.

   The system consists of:

   - Frontend: React-based financial dashboard
   - Backend: API orchestration layer (decision, simulation, data processing)
   - AI Layer: ILMU AI GLM as external reasoning engine
   - Data Layer: ERP + FX market + news ingestion system
   - Simulation Engine: FX impact simulation module (non-numeric estimation)

© UMHackathon 2026

2. **Data Flows:**

   Financial data flows through the system as follows:

   - ERP data (cash, payables, receivables) is collected and structured
   - External FX and news data are fetched and cleaned
   - Both datasets are merged into a unified AI input context
   - ILMU AI GLM processes the input and generates structured financial decisions
   - Output is passed to simulation engine for EX impact simulation (gain/loss/ neutral)
   - Final results are displayed in the decision feed and dashboard UI

3. **Model Process:**

   The system follows an end-to-end AI decision workflow:

   a. User requests financial recommendation
   b. System gathers ERP + market + news data
   c. GLM generates decision (convert/delay/hedge/negotiate)
   d. Explainable AI module generates reasoning
   e. Simulation engine simulates potential financial impact (qualitative estimation)
   f. Results are visualized in dashboard

4. **Role of Reference:**

   This document serves as a technical blueprint for developers, testers, and evaluators to understand the architecture, AI integration, and system bahevior of TreasurAI.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2. Background:

Many multinational companies face significant financial losses due to:

- FX volatility
- Poor timing of currency conversion
- Inefficient cash management
- Lack of real-time decision intelligence

Currently, decisions are made manually using intuition and static reports, which leads to suboptimal financial outcomes.

1. **Previous Version:**

   Traditional treasury systems rely on static dashboards, manual financial analysis, and delayed reporting systems. There is no AI-driven decision layer that actively recommends optimized financial actions in real time.

2. **Changes in Major Architectural Components**

   TreasurAI introduces:

   - AI-driven decision engine (ILMU AI GLM integration)
   - Real-time financial simulation system
   - Explainable AI (XAI) for transparency
   - Dual data intelligence layer (ERP + market data)
   - Fallback intelligence mode for system reliability

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
CFO / Finance Manager	Uses system for financial decision-making	Accurate FX timing, cost savings, risk reduction
Treasury Analyst	Supports financial planning	Clear insights and simulation tools
Development Team	Builds system architecture	Modular, scalable API design
QA Team	Validates system correctness	Reliable AI outputs, correct financial logic
External Data Providers	Provide FX/news/ERP data	Stable and timely data delivery

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Financial Intelligence Dashboard
Architecture	Cloud-based AI-driven system with modular backend services and external AI integration (ILMU AI GLM)

TreasurAI is structured as a client-based architecture enhanced with an AI service layer. The system consists of a single primary client: Finance Web Dashboard (CFO / Treasury User Interface). All user interactions are handled through this web interface, which communicates with a centralized backend API layer.

The backend is composed of multiple logical services:

- Decision service (handles financial recommendation generation)
- Simulation service (runs FX and ROI simulations)
- Data integration service (processes ERP + FX + news data)
- Explainability service (generates reasoning for AI decisions)

These backend services interact with the ILMU AI GLM external API, which acts as the core intelligence engine responsible for generating structured financial decisions.

The system is deployed on a cloud environment, enabling scalability, high availability, and support for real-time financial decision processing.

© UMHackathon 2026

2.1.2 GLM as Service Layer

TreasurAI system integrates ILMU AI GLM as an external AI service layer, rather than a local or embedded AI module. This design ensures scalability, modularity, and separation between financial data processing and AI reasoning.

2.1.3 Dependency Diagram

a. Prompt Construction & GLM API Request Flow

Prompts are dynamically constructed in the backend using structured financial context.

Each prompt includes:

- Company financial state (cash, payables, receivables)
- FX market data (live exchange rates)
- Financial news signals (economic trends, volatility indicators)
- User intent (convert, delay, hedge, negotiate)
- System constraints (risk thresholds, exposure limits)

This structured prompt is then sent to the ILMU AI GLM API via a RESTful request.

b. Context Window Composition

The GLM context window is carefully optimized and contains:

- Current ERP financial snapshot (full data from erp_data.json)
- Latest FX rates (filtered, non-redundant values)
- Relevant financial news (ranked by relevance score)
- User action request (decision type)

© UMHackathon 2026

c. Response Handling & System Processing Flow

Step 1: Response Parsing

- Extract JSON output that includes financial decision, risk level, reasoning, and impact classification.

Step 2: Validation Layer

- Validate scheme correctness
- Ensure financial constraints are not violated
- Reject or fallback invalid outputs

Step 3: Routing Layer

Parsed results are distributed to:

- Decision Feed (frontend UI)
- Explainable AI Module (XAI panel)
- Simulation Engine (ROI simulation system)

d. Token Management & Chunking Strategy

To ensure GLM efficiency and cost control:

- Input data is limited to 3 news items and essential ERP fields
- Keyword filtering removes non-treasury-relevant news headlines
- Cached results are reused for repeated queries
- No automated chunking, ranking, or token counting is performed

© UMHackathon 2026

e. Major API Calls Between System Components

Flows	Details
Frontend → Backend API Gateway	• POST /analyze → Request financial recommendation • POST /simulation → Run scenario analysis • GET / dashboard → Fetch UI financial state
Backend → ILMU AI GLM API	POST /glm/generate • Sends structured financial and market context • Receives JSON decision and reasoning output
Backend → Simulation Engine	Internal Python function calls • fx_simulator.compare_strategies() called directly from analysis_controller.py • roi_calculator.calculate_*() methods called from API routes in api_routes.py • scenario_manager.simulate_*() methods called for scenario simulations
Backend → Data Engine	Store: • Financial records • AI decisions • Simulation outputs • User interaction history
Backend → External APIs	• FX market API → real-time exchange rates • Financial News API → macroeconomic updates • ERP System API → corporate financial data

f. Distributed System Architecture Highlights

TreasurAI is deployed as a monolithic FastAPI application where all backend components run in a single process:

- Frontend (React) handles user interaction only
- Backend API (main.py) manages all orchestration, routing, data services, and simulation logic in one process
- GLM functions as an external AI reasoning service (ILMU AI API)

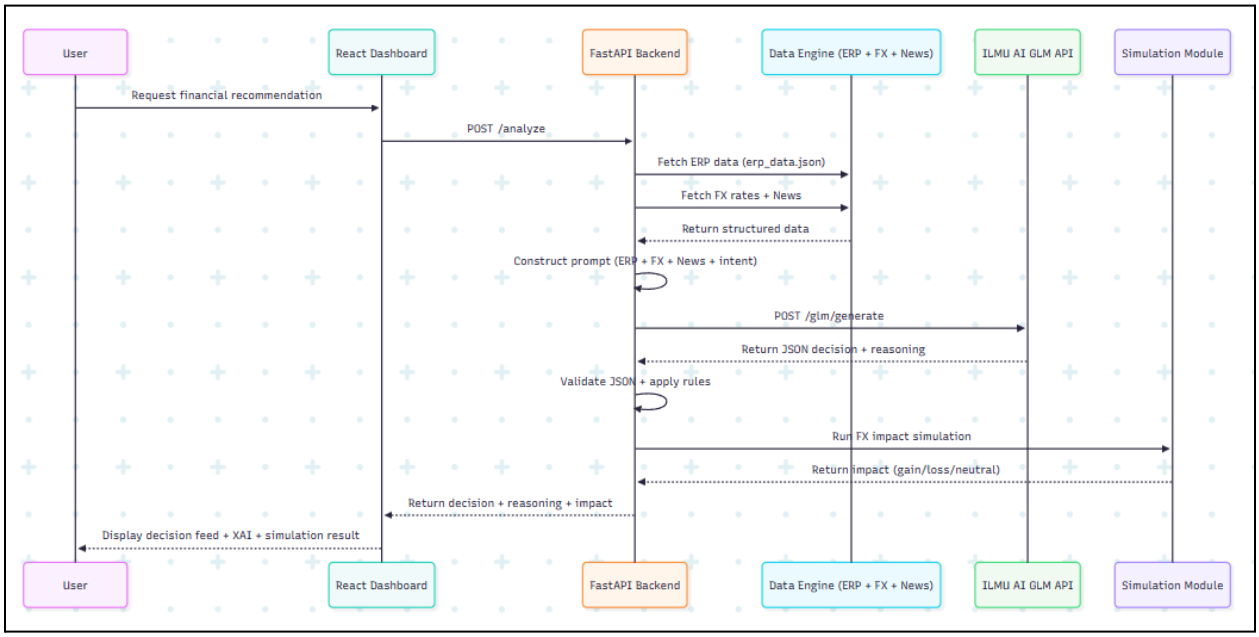
© UMHackathon 2026

- All internal services (data ingestion, decision engine, simulation, cash flow projection) are imported as Python modules within the same application, not deployed as separate micro services.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.4 Sequence Diagram



The sequence diagram shows logical separation of services for clarity. In the actual implementation, the Backend API Gateway, Data Engine, and Simulation Engine all run within a single FastAPI process. Only the GLM is a truly separate external service.

© UMHackathon 2026

2.2. Technological Stack

Layer	Technology	Justification
Frontend	React.js + TypeScript + Tailwind CSS	Enables rapid development of a responsive, interactive financial dashboard with utility-first styling. Supports dynamic UI updates for decision feed, charts, and simulation features.
Backend	FastAPI (Python)	Provides high-performance RESTful APIs with async support. Suitable for integrating AI services, simulation engines, and real-time data processing.
AI Service Layer	ILMU AI GLM API	Core reasoning engine for generating financial decisions, explanations, and trade-off analysis using structured + unstructured data.
Data Integration	External APIs (ERP, FX Rates, Financial News)	Supplies real-time structured (ERP, FX) and unstructured (news) data required for AI decision-making.
Simulation Engine	Python-based Modules	Handles FX simulation, impact estimation, and scenario modeling for decision comparison and impact visualization.
API Communication	RESTful APIs (JSON)	Ensures standardized communication between frontend, backend, AI services, and external data providers.
Development Tools	VS Code + GitHub	Supports collaborative development, version control, and integration with CI/CD pipelines.

2.3. Key Data Flows

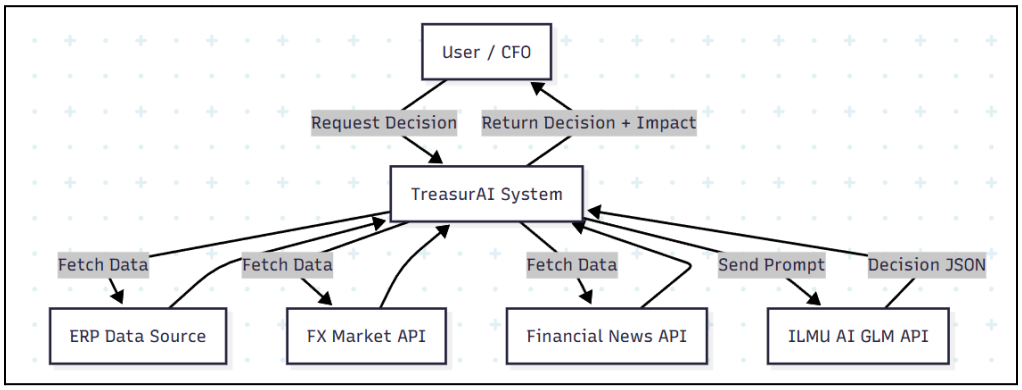
2.3.1. Data Flow Overview

TreasurAI processes both structured data (ERP, FX rates) and unstructured data (financial news) to generate AI-driven financial decisions. The data flows through multiple system layers, ensuring proper validation, transformation, and storage before being presented to users.

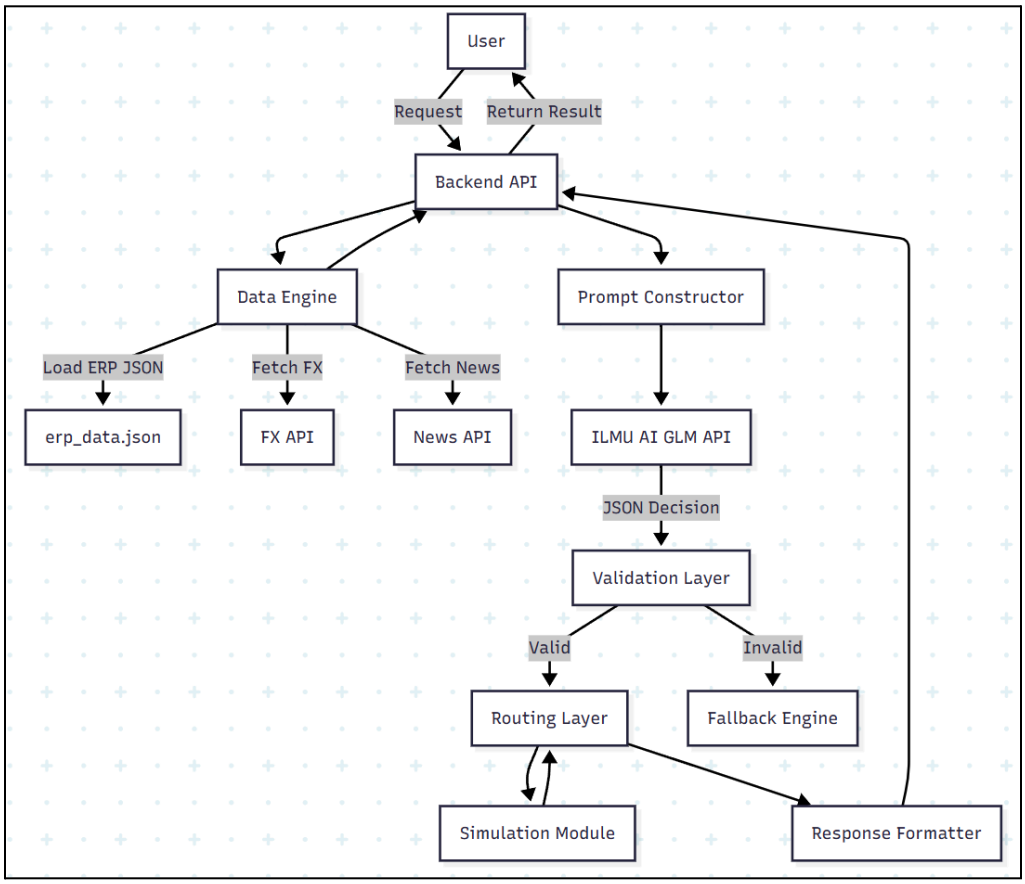
© UMHackathon 2026

2.3.1.1. Data Flow Diagram (DFD)

DFD Level 0



DFD Level 1



© UMHackathon 2026

Email: umhackathon@um.edu.my

DFD Validation

- **No Block Hole:** Every input (user request) produces output (decision or fallback response)
- **No Gray Hole:** All processes have sufficient input data before producing output
- **No Data Sink Issue:** All generated data is stored or displayed (no loss of critical data)

2.3.2. Normalized Database Schema

TreasurAI does not use a database. All data is stored and managed as follows:

- **ERP data:** Loaded from erp_data.json file at runtime
- **Cache:** In-memory Python dictionary with 5-minute TTL
- **News data:** Fetched live from RSS feeds on each request
- **GLM decisions:** Generated on-demand, not persisted

© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	AI Decision Feed	Generates real-time financial recommendations such as currency conversion, payment delay, supplier negotiation, and hedging strategies based on ERP data and market signals.
2	Explainable AI (XAI)	Provides detailed reasoning for each decision, including financial data used, relevant news signals, and logical explanation to ensure transparency and trust.
3	Trade-off & ROI Simulator	Allows users to compare multiple options (convert now vs wait vs hedge) and view potential impact, risk levels, and decision comparison.
4	Interactive Cash Flow Visualization	Displays financial timeline with before-and-after comparison using chart overlays, helping users understand the impact of selected decisions.
5	Fallback Intelligence Mode	Ensures system reliability by providing alternative rule-based recommendations when AI service (GLM) is unavailable.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	System must handle multiple concurrent users and real-time financial data updates	Deploy on cloud infrastructure with scalable backend services
Reliability	System must provide consistent recommendations and avoid failure in decision generation	Implement fallback mode and error handling mechanisms
Performance	API response time should be <800ms (excluding AI response)	Use asynchronous API calls and optimized data processing
Maintainability	System components should be modular and independently updatable	Separate backend services (data, AI, simulation)
Token Latency	GLM response should be within acceptable time (≤ 3–5 seconds)	Apply prompt optimization and token control strategies
Cost Efficiency	AI usage cost must be controlled	Use prompt truncation, caching, and data filtering before GLM calls

3.3. Out of Scope / Future Enhancements

- Real-time trade execution with banking integration
- Multi-user collaboration and role-based workflows
- Advanced hedging instruments
- Mobile application version
- Predictive long-term financial forecasting using ML models

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

TreasurAI adopts a gradual rollout strategy to ensure stability before full deployment. New AI decision features are initially released to a small percentage of users, where system performance, accuracy of recommendations, and user interactions are monitored. Once validated, the feature is progressively expanded to all users.

A/B testing is conducted to compare:

- Different decision strategies (e.g., aggressive vs conservative recommendations)
- UI interaction designs (e.g., decision feed layout, button placement)

4.1.2. Emergency Rollback (ER) & Golden Release:

In case of system failure or abnormal AI behavior:

- Emergency Rollback is triggered
- System reverts to a stable “Golden Version” of the application

Fallback mode is activated to ensure continuity:

- Rule-based recommendations replace AI outputs
- Users are notified of reduced intelligence mode

© UMHackathon 2026

4.1.3. Priority Matrix:

Priority Level	Condition	Action
P1 (Critical)	GLM API failure or invalid decision output	Activate fallback mode + trigger rollback
P2 (High)	API response delay >5s or simulation failure	Retry request + notify user
P3 (Medium)	Minor UI or data inconsistency	Log issue and fix in next release
P4 (Low)	Cosmetic issues or non-critical bugs	Monitor and resolve during maintenance

4.2. Monitoring:

The system continuously monitors:

- API response time
- GLM success/failure rate
- Decision accuracy (based on simulation outcomes)
- System uptime and error rates

Alerts are triggered when:

- Response time exceeds threshold
- AI output fails validation
- Data pipeline description occurs

4.3. Assumptions:

- Users have basic financial knowledge
- ERP data provided is accurate and up-to-date
- External APIs (FX rates, news) are reliable
- Users have stable internal connection for real-time interaction
- GLM API is available under normal operating conditions

© UMHackathon 2026

4.4. External Dependencies:

Tools	Purpose	Risks
ILMU AI GLM API	Core AI reasoning engine for decision-making and explanation generation	High – System heavily depends on GLM availability
FX Market API	Provides real-time exchange rates	Medium – Delay or inaccuracy affects decisions
Financial News API	Supplies market signals and economic context	Medium – Irrelevant or noisy data may affect reasoning
ERP System API	Provides company financial data	High – Incorrect data leads to wrong decisions
Cloud Services (AWS)	Hosting, data engine, and deployment infrastructure	Medium – Downtime affects system availability

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Project Management & Team Contributions

5.1. Project Timeline:

Day	Phase	Activities
1-2	Planning & Ideation	Define problem statement, finalize features, assign team roles, design system architecture
3-4	UI/UX Development	Design and build frontend using Figma and React (dashboard, decision feed, charts)
5-6	Backend Development	Develop API endpoints, integrate data services, connect frontend with backend
7-8	AI & Simulation Integration	Implement GLM decision engine, simulation engine, impact simulation logic
9	Testing & Optimization	Debug system, test workflows, improve performance and reliability
10	Finalization & Presentation	Prepare documentation (SAD, PRD, QATD), demo video, and pitch

5.2. Team Members & Role:

Member	Role	Responsibilities
Chua Xin Yi	Frontend and UI Development	Build dashboard UI, decision feed, charts, user interaction flow
Lee Zi Xuan	GLM Decision Engine	Develop GLM integration, prompt engineering, decision logic, explainable AI output
Ho Kang Zheng	Data Integration	Build ERP mock data, integrate FX + news APIs, data cleaning and preprocessing
Chong Xiao Qing	Backend and System Orchestrator	Develop API endpoints, system pipeline, fallback handling, backend integration
Lee Qian Yi	Simulation and ROI	Implement FX simulation, scenario modeling, impact analysis

© UMHackathon 2026

Email: umhackathon@um.edu.my

5.3. Recommendations:

- Implement Redis caching to reduce repeated GLM API calls
- Introduce WebSocket for live decision feed
- Use Docker containerization for easier deployment and scalability
- Enhance security with API authentication and access control
- Optimize prompt design to reduce token cost and latency

© UMHackathon 2026